

CS121 Data Structures in C

10 Priority Queue (Heaps) & Heap Sort

Outline

- Priority queue
- Background knowledge
- Heaps
- Heap sort

Motivating Example

3 jobs have been submitted to a printer in the order A, B, C.

Sizes: Job A – 100 pages

Job B – 10 pages

Job C -- 1 page

Average waiting time with FIFO service:

$$(100+110+111) / 3 = 107 \text{ time units}$$

Average waiting time for shortest-job-first service:

$$(1+11+111) / 3 = 41 \text{ time units}$$



A queue be capable to **insert** and **deletemin**?

Priority Queue

Priority Queue

- Priority queue is a data structure which allows at least two operations
 - ♦ **Insert**: equivalent of **Enqueue**
 - ♦ **DeleteMin**: finds, returns and removes the minimum elements in the priority queue



- Applications: O.S., external sorting, greedy algorithms

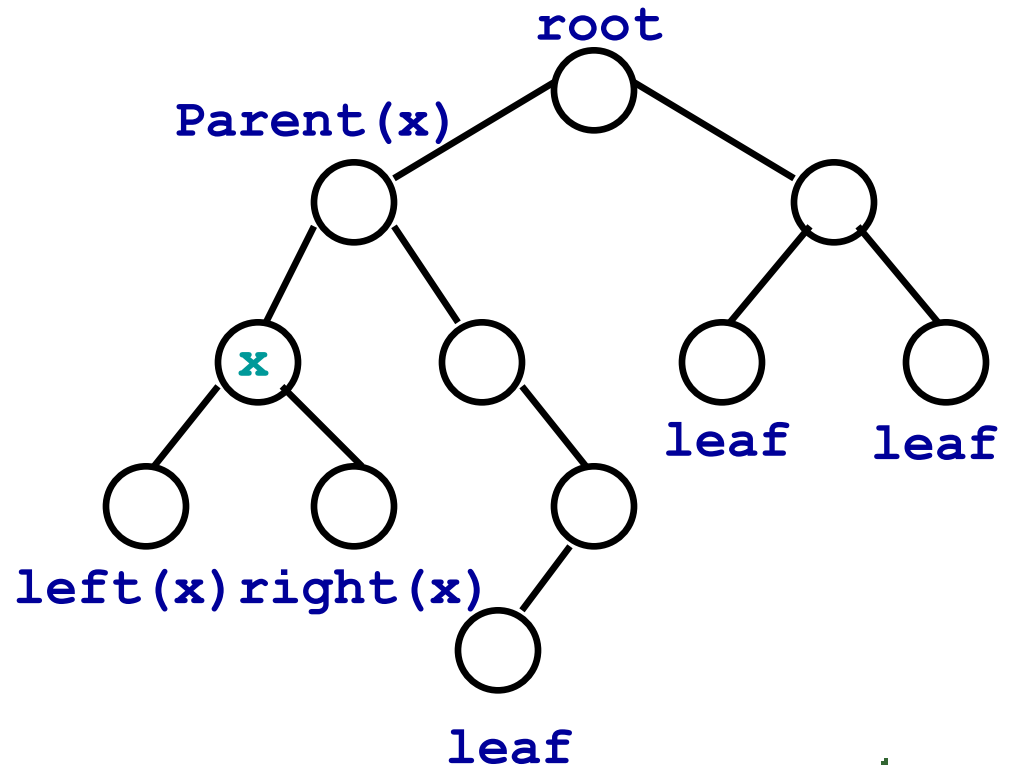
Possible Implementations

- **Linked list**
 - ♦ Insert in $O(1)$
 - ♦ Find the minimum element in $O(N)$, thus deleteMin is $O(N)$
- **Binary search tree (AVL tree)**
 - ♦ Insert in $O(\log N)$
 - ♦ Delete in $O(\log N)$
 - ♦ Repeatedly removing a node in the left subtree -- unbalanced
 - ♦ Search tree is an overkill as it supports a host of operations that are not required
- Eerr, neither fit quite well...
- How about another type of tree?



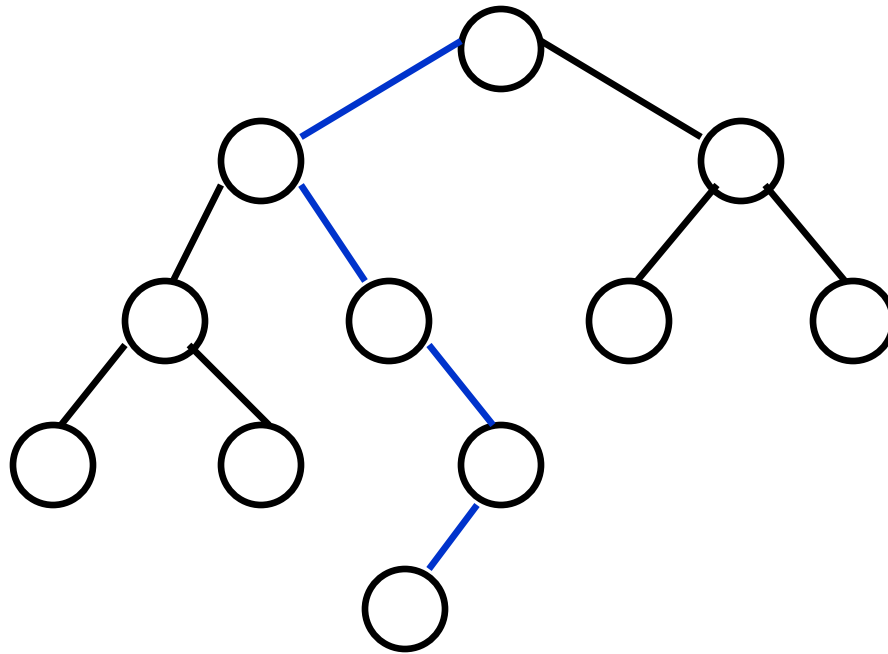
Background: Binary Trees

- Has a root at the topmost level
- Each node has zero, one or two children
- A node that has no child is called a leaf
- For a node x , we denote the left child, right child and the parent of x as $\text{left}(x)$, $\text{right}(x)$ and $\text{parent}(x)$, respectively.



Background: Height (Depth) of A Binary Tree

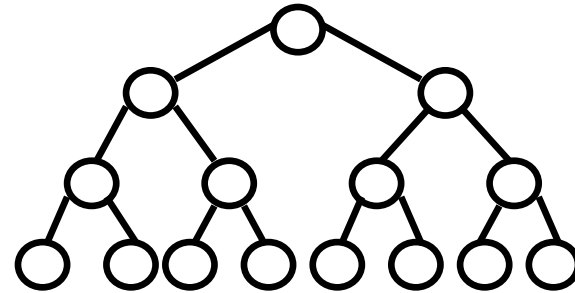
- The number of edges on the **longest** path from the root to a leaf.



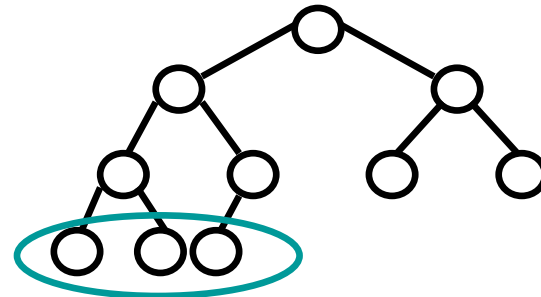
Height = 4

Complete Binary Tree

- Full Binary Tree (Perfect Binary Tree)
 - ♦ A *full binary tree* in which all *leaves* are at the same *depth*.



- Almost (nearly) Complete Binary Tree
 - ♦ A tree is completely filled, with the possible *exception of the bottom level*, which is filled from *left to right*.

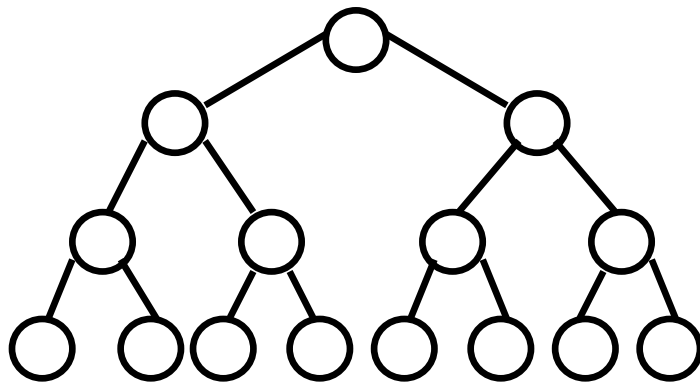


Complete Binary Tree (Perfect Binary Tree)

- Properties:

- ♦ The number of leaf nodes with height h : 2^h

- ♦ The number of nodes with height h : $\sum_{i=0}^h 2^i = 2^{h+1} - 1$

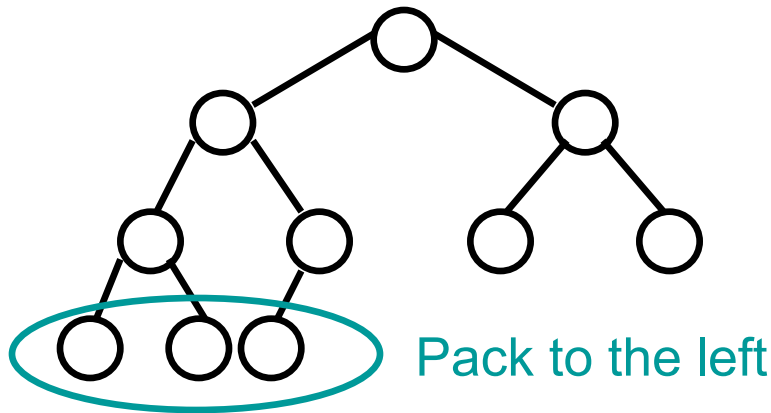


height	no. of leaf nodes
0	1
1	2
2	4
3	8
h	2^h

- ♦ A complete binary tree with N nodes has height: $O(\log N)$

Complete Binary Tree (Almost)

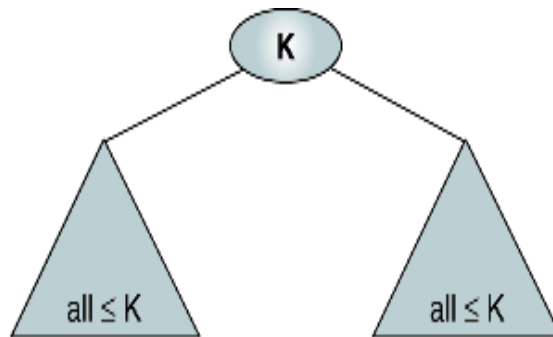
- All levels are full, except the last one, which is left-filled.



- Properties:
 - ◆ The number of nodes with height h : $2^h \sim 2^{h+1}-1$

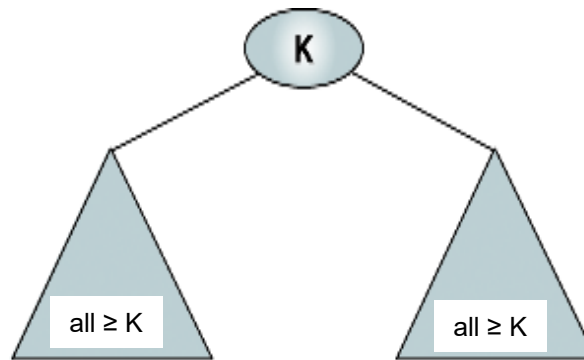
Basic Concept of Heap

- Binary Heap (Max-Heap)
 - ♦ A binary tree whose **left and right subtrees** have values *less than or equal to* their **parents**.
- With following properties
 - ♦ The tree is **complete** or **nearly complete**.
 - ♦ The key value of each node is greater than or equal to the key value in each of its descendents.



Basic Concept of Heap

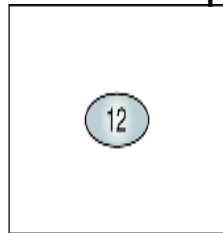
- Binary Heap (Min-Heap)
 - ♦ A binary tree whose **left and right subtrees** have values *more than or equal to* their **parents**.
- With following properties
 - ♦ The tree is **complete** or **nearly complete**.
 - ♦ The key value of each node is smaller than or equal to the key value in each of its descendants.



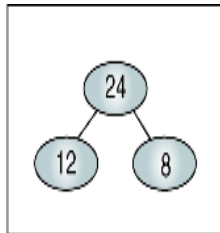
Heap

- Example

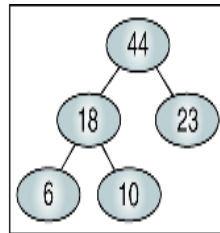
Max-Heap



(a) Root-only heap



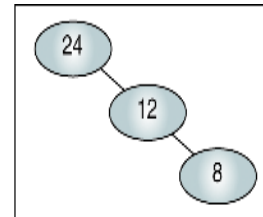
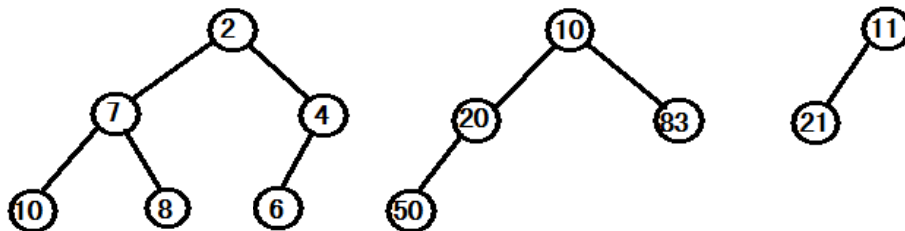
(b) Two-level heap



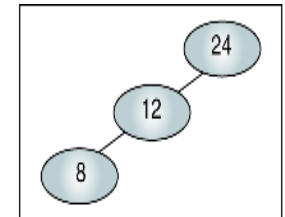
(c) Three-level heap



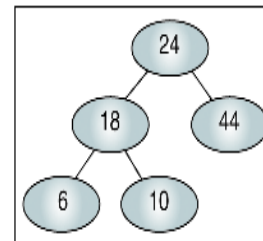
Min-Heap



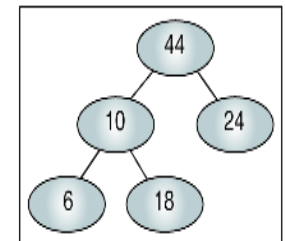
(a) Not nearly complete (rule 1)



(b) Not nearly complete (rule 1)



(c) Root not largest (rule 2)



(d) Subtree 10 not a heap (rule 2)



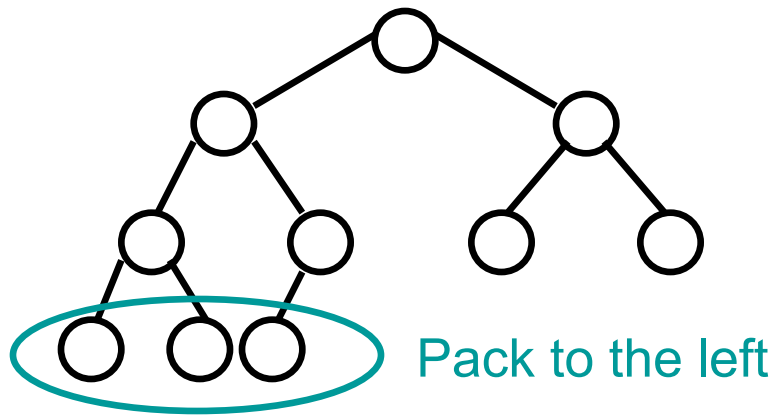
What Are Heaps Useful for?

- To implement **priority queues**
- Priority queue = a queue where all elements have a “priority” associated with them
- Which allows at least two operations
 - ◆ Insert
 - ◆ DeleteMin: finds, returns and removes the minimum elements (smallest priority) in the priority queue

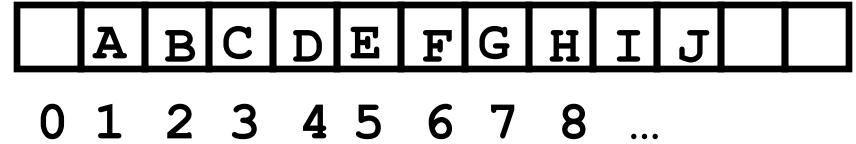
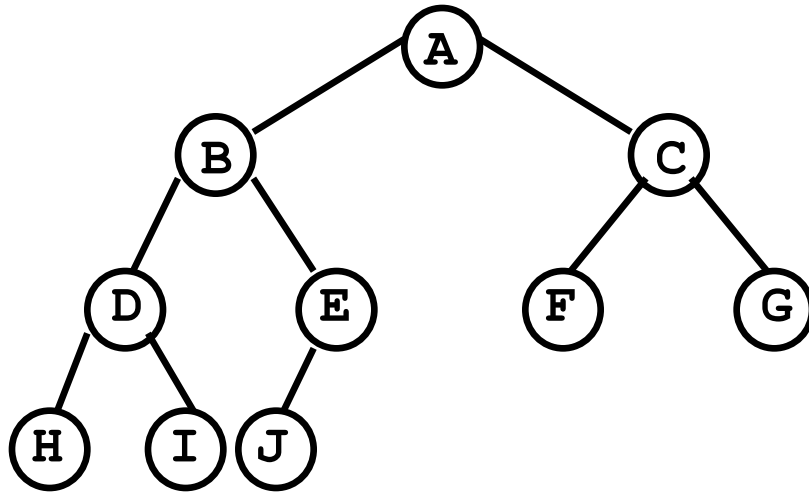


Implementation of Binary Heap

- Structure properties
 - ♦ Has 2^h to $2^{h+1}-1$ nodes with height h
 - ♦ The structure is so regular, it can be represented in an **array** and no links are necessary!



Array Implementation of Binary Heap

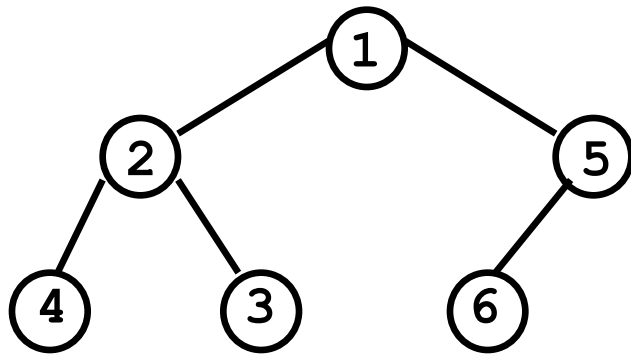


- For any element in **array position i**
 - ♦ The **left child** is in position **$2i$**
 - ♦ The **right child** is in position **$2i+1$**
 - ♦ The **parent** is in position **$\text{floor}(i/2)$**
- A possible problem: an estimate of the maximum heap size is required in advance (but normally we can resize if needed).

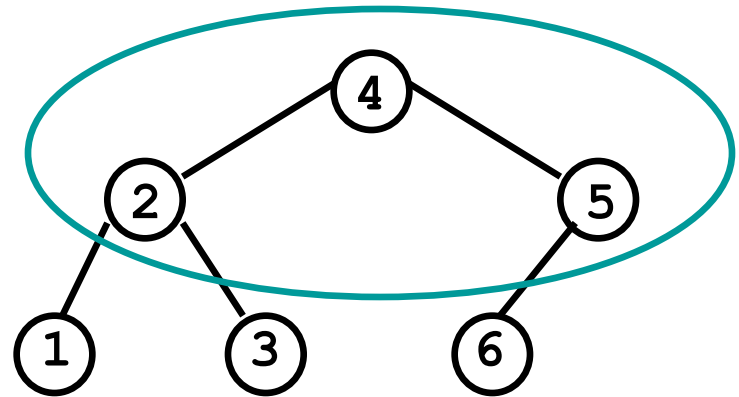


Heap Order Property

- Heap-order property (Min-Heap): the value at each node is **less than or equal** to the values at both its descendants.
 - ♦ It is easy (both conceptually and practically) to perform **insert** and **deleteMin** in heap if the heap-order property is maintained.
- Use of binary heap is so common for priority queue implementations, thus the word heap is usually assumed to be the implementation of the data structure.



A heap



Not a heap

Heap Properties

- Heap supports the following operations efficiently
 - ◆ Insert in $O(\log N)$ time
 - ◆ Locate the current minimum in $O(1)$ time
 - ◆ Delete the current minimum in $O(\log N)$ time

Array Implementation of Heap

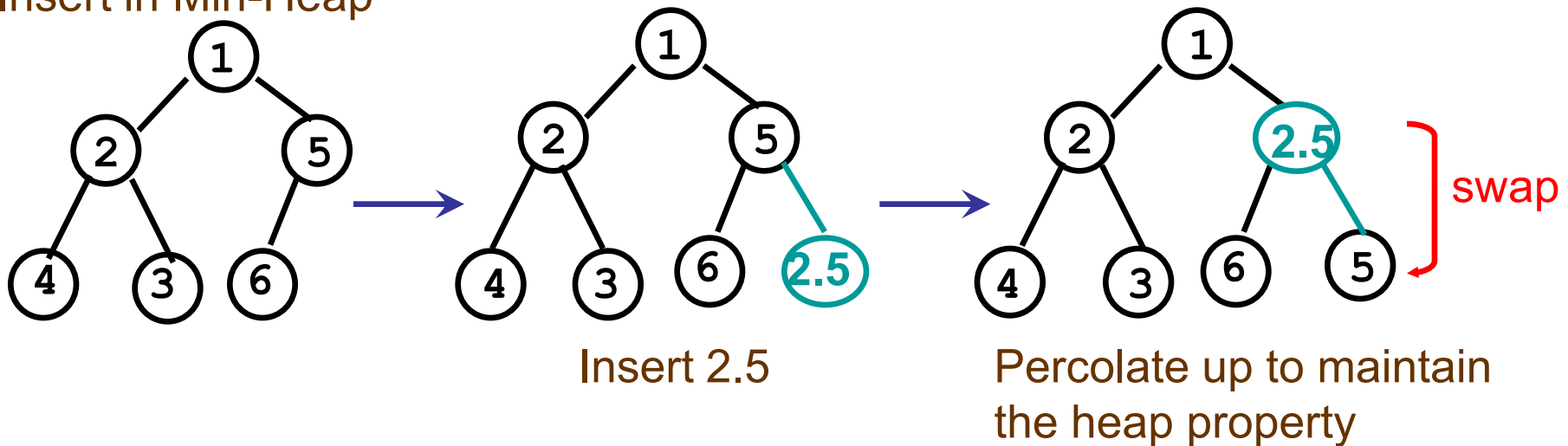
- Operations
 - ◆ Creation of an empty heap
 - ◆ Insertion of a new element into the heap
 - ◆ Deletion of the largest (smallest) element from the heap
- Heap is complete binary tree, can be represented by array. So the complexity of inserting any node or deleting the root node from Heap is $O(\log N)$.

Insertion

- Algorithm

1. Add the new element to the next available position at the lowest level.
2. Restore the min-heap property if violated.
 - General strategy is **percolate up** (or bubble up): if the parent of the element is larger than the element, then interchange the parent and child.

Insert in Min-Heap



Implementation of Heap

- Declaration for priority queue

```
typedef int ElementType;  
  
struct HeapStruct;  
typedef struct HeapStruct *PriorityQueue;  
  
PriorityQueue Initialize( int MaxElements );  
void Destroy( PriorityQueue H );  
void MakeEmpty( PriorityQueue H );  
void Insert( ElementType X, PriorityQueue H );  
ElementType DeleteMin( PriorityQueue H );  
ElementType FindMin( PriorityQueue H );  
int IsEmpty( PriorityQueue H );  
int IsFull( PriorityQueue H );
```

```
struct HeapStruct  
{  
    int Capacity;  
    int Size;  
    ElementType *Elements;  
};
```

Implementation of Heap (cont'd)

- Initialization a priority queue – create a new one

```
#define MinPQSize (10)
#define MinData (-32767)

PriorityQueue
Initialize( int MaxElements )
{
    PriorityQueue H;

    if( MaxElements < MinPQSize )
        Error( "Priority queue size is too small" );

    H = malloc( sizeof( struct HeapStruct ) );
    if( H ==NULL )
        FatalError( "Out of space!!!" );

    /* Allocate the array plus one extra for sentinel */
    H->Elements = malloc( ( MaxElements + 1 )
                          * sizeof( ElementType ) );
    if( H->Elements == NULL )
        FatalError( "Out of space!!!" );

    H->Capacity = MaxElements;
    H->Size = 0;
    H->Elements[ 0 ] = MinData;

    return H;
}
```

Implementation of Heap (cont'd)

- Insert elements into the Min-heap

```
/* H->Element[ 0 ] is a sentinel */
```

```
void
```

```
Insert( ElementType X, PriorityQueue H )
```

```
{
```

```
    int i;
```

```
    if( IsFull( H ) )
```

```
    {
```

```
        Error( "Priority queue is full" );
```

```
        return;
```

```
    }
```

```
    for( i = ++H->Size; H->Elements[ i / 2 ] > X; i /= 2 )
```

```
        H->Elements[ i ] = H->Elements[ i / 2 ];
```

```
    H->Elements[ i ] = X;
```

```
}
```

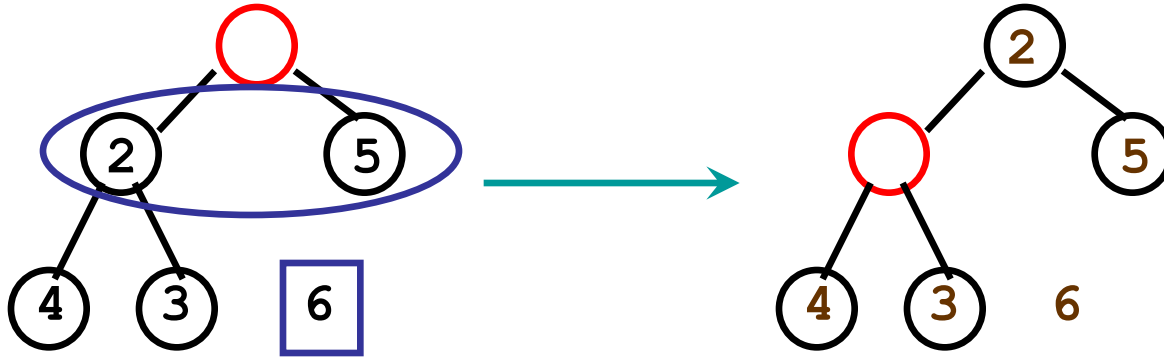
Deletion

- Delete the minimum element
- First attempt

Algorithm

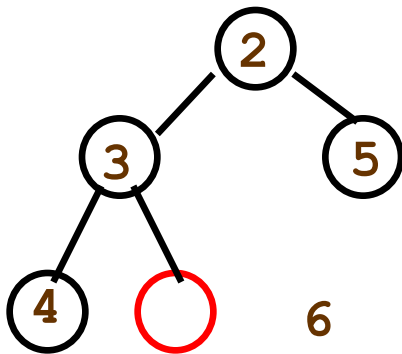
1. Delete the root.
2. Compare the two children of the root
3. Make the lesser of the two the root.
4. An empty spot is created.
5. Bring the lesser of the two children of the empty spot to the empty spot.
6. A new empty spot is created.
7. Continue

Deletion with 'Hole Trick'



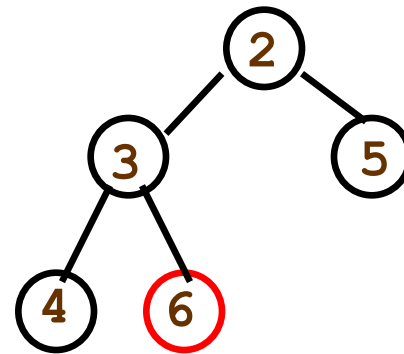
1. create hole
tmp = 6 (last element)

2. Compare children and tmp
bubble down if necessary



3. Continue step 2 until
reaches lowest level

4. Fill the hole



Implementation

- Perform DeleteMin in a binary heap

```
ElementType
DeleteMin( PriorityQueue H )
{
    int i, Child;
    ElementType MinElement, LastElement;

    /* 1*/    if( IsEmpty( H ) )
    {
        /* 2*/        Error( "Priority queue is empty" );
        /* 3*/        return H->Elements[ 0 ];
    }

    /* 4*/    MinElement = H->Elements[ 1 ];
    /* 5*/    LastElement = H->Elements[ H->Size-- ];
```

~ go to the next page ~

Implementation (cont'd)

```
/* 6*/      for( i = 1; i * 2 <= H->Size; i = Child )
            {
                /* Find smaller child */
/* 7*/      Child = i * 2;
/* 8*/      if( Child != H->Size && H->Elements[ Child + 1 ]
/* 9*/                      < H->Elements[ Child ] )
/*10*/          Child++;

                /* Percolate one level */
/*11*/      if( LastElement > H->Elements[ Child ] )
/*12*/          H->Elements[ i ] = H->Elements[ Child ];
                else
/*13*/          break;
            }
/*14*/      H->Elements[ i ] = LastElement;
/*15*/      return MinElement;
    }
```

Conclusion

- Priority queue
 - ♦ A queue where all elements have a priority associated with them
 - ♦ At least two operations
 - Insert
 - DeleteMin
 - ♦ Binary heap (Heap) is used for implementing priority queue
 - Array implementation

Assignment 10&11

- Asg10: Written: Textbook 6.2(a), 6.3

Submit it to Moodle

- Asg11:

Write and test a program that performs the operations Insert, DeleteMin, BuildHeap, FindMin in a binary heap (MinHeap).

- ♦ Input 53, 26, 97, 59, 41, 31, 58 to do the test.
 - To construct the binary heap and print it;
 - Find the minimum element from the heap and print it;
 - Delete the minimum one and then print the result heap after the deletion.

Submit it to Moodle

Attach your BinHeap.c and BinHeap.h

Heap Sort

Heap Sort

- The heap sort is an improved version of the selection sort in which the largest/smallest element (the root) is selected and exchanged with the last element in the unsorted list.
- Input: N elements
- Output: A heap with heap-order property
 - (1) Build a binary heap of N elements
 - the minimum element is at the top of the heap
 - (2) Perform N **DeleteMin** operations
 - the elements are extracted in sorted order
 - (3) Record these elements in a second array and then copy the array back.

Heap Sort – Running Time Analysis

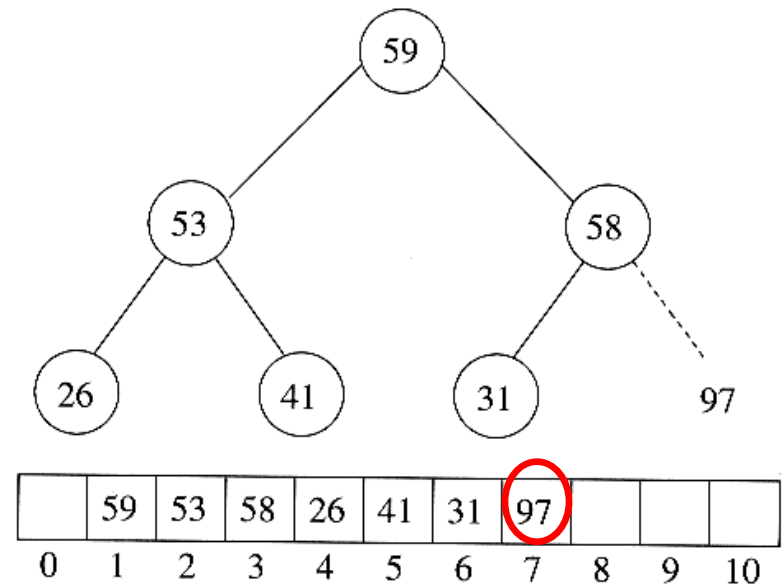
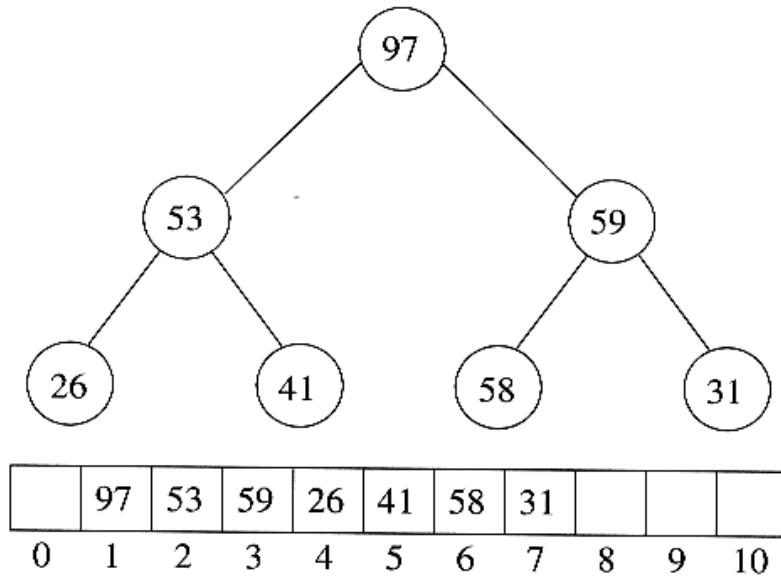
- (1) Build a binary heap of N elements
 - ♦ Repeatedly insert N elements $\Rightarrow O(N \log N)$ time
 - (2) Perform N **DeleteMin** operations
 - ♦ Each **DeleteMin** operation takes $O(\log N) \Rightarrow O(N \log N)$
 - (3) Record these elements in a second array and then copy the array back
 - ♦ $O(N)$
- Total time complexity: $O(N \log N)$
 - Memory requirement: uses an extra array, $O(N)$

Heap Sort: No Extra Storage

- Observation: after each **deleteMin**, the size of heap shrinks by 1
 - ♦ We can use **the last cell** just freed up to store the element that was just deleted
 - ⇒ after the last **deleteMin**, the array will contain the elements in decreasing sorted order
- To sort the elements in the decreasing order, use a min heap
- To sort the elements in the increasing order, use a max heap
 - ♦ the parent has a larger element than the child

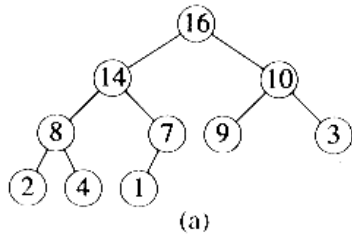
Heap Sort Example 1: No Extra Storage

Sort in increasing order: use max heap

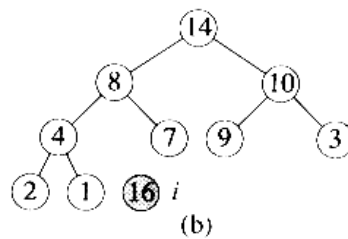


Heap Sort Example 2: No Extra Storage

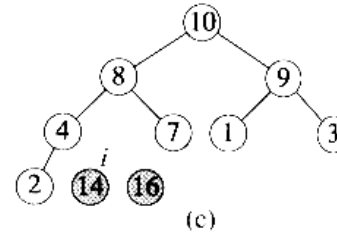
Delete 16



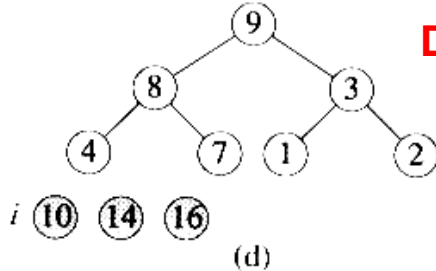
Delete 14



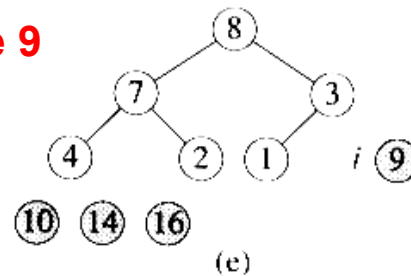
Delete 10



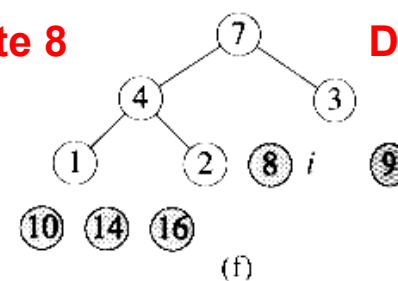
Delete 9



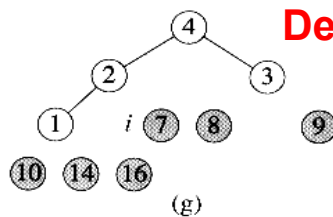
Delete 8



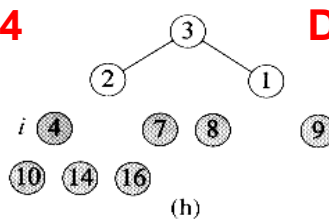
Delete 7



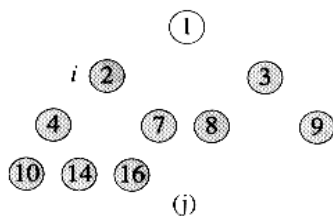
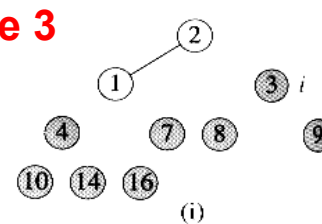
Delete 4



Delete 3



Delete 2



A

1	2	3	4	7	8	9	10	14	16
---	---	---	---	---	---	---	----	----	----

(k)